

1 Syntax

$\diamond_u$	$::=$	UnOpType
$\diamond_b$	$::=$	BinOpType
$\diamond_r$	$::=$	RelOpType
$\diamond_c$	$::=$	CastKind
var	$::=$	RegVar($k, reg_{id}, name$) PCVar(k) TempVar(k, id) StackVar(k, id) GlobalVar(k, id) MemVar
exp	$::=$	BinOp($\diamond_b, k, exp, exp$) Cast($\diamond_c, k, exp$) ExprList(exp^*) Extract(exp, k, pos) FuncName(s) ITE(exp_c, k, exp_t, exp_f) Load(var_{mem}, k, exp) Store($var_{mem}, k, exp_a, exp_v$) Num(n) RelOp($\diamond_r, k, exp, exp$) UnOp($\diamond_u, k, exp$) Undefined(k, str) Var(var)
$stmt$	$::=$	LMark(ℓ) Def(var, exp) Phi($var, \mathbb{Z}^*$) Jmp($jmpType$) ExternalCall($exp, var_{in}^*, var_{out}^*$) SideEffect($SideEffect$)
$jmpType$	$::=$	IntraJmp(ℓ) IntraCJmp(exp_c, ℓ_t, ℓ_f) InterJmp(exp) InterCJmp(exp_c, exp_t, exp_f)

2 Abstract Domains

(Abstract Value)	$\mathbb{V}$	$=$	$n \mid \perp \mid \top$
(Register Indicator)	$\mathbb{R}$		
(Register Map)	$\mathbb{R}_M$	$=$	$\mathbb{R} \rightarrow \mathbb{V}$
(Location)	$\mathbb{L}$	$=$	$\langle \mathbb{N}, \mathbb{N} \rangle : \langle Version, Address \rangle$
(Memory)	$\mathbb{M}$	$=$	$\mathbb{L} \rightarrow \langle \mathbb{V}, \mathbb{N} \rangle : \langle AbsValue, TypeIndicator \rangle$
(Type Indicator Map)	$\mathbb{T}_I$	$=$	$\mathbb{R} \rightarrow \mathbb{N}$
(Type Constraint)	$\mathbb{T}_C$	$=$	Address($\mathbb{N}$) Value($\mathbb{N}$) Same($2^{\mathbb{N}}$) AddResult($\mathbb{N} \times \mathbb{N} \times \mathbb{N}$) SubResult($\mathbb{N} \times \mathbb{N} \times \mathbb{N}$)
(Type State)	$\mathbb{T}_S$	$=$	$\langle \mathbb{T}_I \times 2^{\mathbb{T}_C} \rangle$
(Abstract State)	$\mathbb{S}$	$=$	$\mathbb{R}_M \times \mathbb{M} \times \mathbb{T}_S$

3 Summary Domain

(Argument Map)	$\mathbb{A} = \mathbb{Z} \rightarrow \mathbb{V}$
(Operand Type Map)	$\mathbb{O} = instr \rightarrow operand$
(Function Summary)	$\Sigma_F = \langle A, Ret \rangle$
(Return Value of Summarize)	$\Sigma = \langle \Sigma_F, \mathbb{O}, \mathbb{T}_M \rangle$

4 Auxiliary Functions Need to Modify

$$\text{phiVals}(l, S) = \{v_0, \dots, v_n\}$$

$$\text{kind}(l, e) = \text{Call}(f) \mid \text{Return} \mid \text{InterJump}$$

$$\text{args}(l) = \text{callArguments}(l) \quad \text{rets}(l) = \text{callReturns}(l)$$

$$\text{B2R2Const}(l, x) = n \quad \text{iff B2R2 SSA constant propagation reports } x \text{ at } l \text{ as } \text{Const}(n)$$

5 Type Constraint Addition

$$\frac{S = \langle R_M, M, \langle T_I, T_C \rangle \rangle}{\text{add}_{\text{Const}}(S, c) \Downarrow \langle R_M, M, \langle T_I, T_C \cup \{c\} \rangle \rangle}$$

6 Type Constraint Solving

$$\frac{\text{Same}(\{id_0, \dots, id_n\}) \in T_C \quad \forall i. \text{Address}(id_i) \notin T_C \wedge \text{Value}(id_i) \notin T_C}{T'_M = T_M}$$

$$\frac{\text{Same}(\{id_0, \dots, id_n\}) \in T_C \quad \exists i \forall j. \text{Address}(id_i) \in T_C \wedge \text{Value}(id_j) \notin T_C}{T'_C = T_C \cup \{\text{Address}(id_i) \mid i \in \{0, \dots, n\}\}}$$

$$\frac{\text{Same}(\{id_0, \dots, id_n\}) \in T_C \quad \exists i \forall j. \text{Value}(id_i) \in T_C \wedge \text{Address}(id_j) \notin T_C}{T'_C = T_C \cup \{\text{Value}(id_i) \mid i \in \{0, \dots, n\}\}}$$

$$\frac{\text{AddResult}(id_d, id_0, id_1) \in T_C \quad \text{Address}(id_0) \in T_C \quad \text{Value}(id_1) \in T_C}{T'_C = T_C \cup \{\text{Address}(id_d)\}}$$

$$\frac{\text{AddResult}(id_d, id_0, id_1) \in T_C \quad \text{Value}(id_0) \in T_C \quad \text{Address}(id_1) \in T_C}{T'_C = T_C \cup \{\text{Address}(id_d)\}}$$

$$\frac{\text{AddResult}(id_d, id_0, id_1) \in T_C \quad \text{Value}(id_0) \in T_C \quad \text{Value}(id_1) \in T_C}{T'_C = T_C \cup \{\text{Value}(id_d)\}}$$

$$\frac{\text{AddResult}(id_d, id_0, id_1) \in T_C \quad \text{Address}(id_d) \in T_C \quad \text{Value}(id_0) \in T_C}{T'_C = T_C \cup \{\text{Address}(id_1)\}}$$

$$\frac{\text{AddResult}(id_d, id_0, id_1) \in T_C \quad \text{Address}(id_d) \in T_C \quad \text{Value}(id_1) \in T_C}{T'_C = T_C \cup \{\text{Address}(id_0)\}}$$

$$\frac{\text{AddResult}(id_d, id_0, id_1) \in T_C \quad \text{Value}(id_d) \in T_C}{T'_C = T_C \cup \{\text{Value}(id_0), \text{Value}(id_1)\}}$$

$$\frac{\text{SubResult}(id_d, id_0, id_1) \in T_C \quad \text{Address}(id_0) \in T_C \quad \text{Value}(id_1) \in T_C}{T'_C = T_C \cup \{\text{Address}(id_d)\}}$$

$$\frac{\text{SubResult}(id_d, id_0, id_1) \in T_C \quad \text{Address}(id_0) \in T_C \quad \text{Address}(id_1) \in T_C}{T'_C = T_C \cup \{\text{Value}(id_d)\}}$$

$$\frac{\text{SubResult}(id_d, id_0, id_1) \in T_C \quad \text{Value}(id_0) \in T_C}{T'_C = T_C \cup \{\text{Value}(id_d), \text{Value}(id_1)\}}$$

$$\frac{\text{SubResult}(id_d, id_0, id_1) \in T_C \quad \text{Address}(id_d) \in T_C}{T'_C = T_C \cup \{\text{Address}(id_0), \text{Value}(id_1)\}}$$

7 AbsVal BinOp

$$\text{updateBinOp}(\diamond_b, S, v_1, v_2, tid_1, tid_2) \Downarrow \langle S', v_r, tid_r \rangle$$

$$\frac{\begin{array}{l} \diamond_b = \text{ADD} \quad S = \langle R, M, \langle T_I, T_C \rangle \quad tid_0, T'_I = \text{newId}(T_I) \\ v_r = v_1 +^\# v_2 \quad T'_C = T_C \cup \{\text{AddResult}(tid_0, tid_1, tid_2)\} \quad S' = \langle R, M, \langle T'_I, T'_C \rangle \rangle \end{array}}{\text{updateBinOp}(S, \diamond_b, v_1, v_2, tid_1, tid_2) \Downarrow \langle S', v_r, tid_0 \rangle}$$

$$\frac{\begin{array}{l} \diamond_b = \text{SUB} \quad S = \langle R, M, \langle T_I, T_C \rangle \quad tid_0, T'_I = \text{newId}(T_I) \\ v_r = v_1 -^\# v_2 \quad T'_C = T_C \cup \{\text{SubResult}(tid_0, tid_1, tid_2)\} \quad S' = \langle R, M, \langle T'_I, T'_C \rangle \rangle \end{array}}{\text{updateBinOp}(S, \diamond_b, v_1, v_2, tid_1, tid_2) \Downarrow \langle S', v_r, tid_0 \rangle}$$

$$\frac{\diamond_b \notin \{\text{ADD}, \text{SUB}\} \quad v_r = v_1 \diamond_b^\# v_2}{\text{updateBinOp}(S, \diamond_b, v_1, v_2, tid_1, tid_2) \Downarrow \langle S, v_r, \text{None} \rangle}$$

8 Expr Evaluation

$$eval(S, expr) \Downarrow \langle v, tid, S' \rangle$$

8.1 BinOp

$$\frac{eval(S, e_1) \Downarrow \langle v_1, tid_1, S_1 \rangle \quad eval(S_1, e_2) \Downarrow \langle v_2, tid_2, S_2 \rangle \quad updateBinOp(\diamond_b, S_1, v_1, v_2, tid_1, tid_2) \Downarrow \langle S_2, v, tid \rangle}{eval(S, BinOp(\diamond_b, k, e_1, e_2)) \Downarrow \langle v, tid, S_2 \rangle}$$

8.2 Store

$$\frac{\begin{array}{l} eval(S, e_a) \Downarrow \langle v_a, tid_a, S_1 \rangle \quad S_1 = \langle R, M, \langle T_I, T_C \rangle \rangle \quad T'_C = T_C \cup \{Address(tid_a)\} \\ eval(\langle R, M, \langle T_I, T'_C \rangle, e_v \rangle \Downarrow \langle v, S_3 \rangle \quad S_3 = \langle R', M', \langle T'_I, T''_C \rangle \rangle \quad tid_e, T'_I = newId(T'_I) \\ M'' = \begin{cases} M'[\langle m.Identifier, v_a \rangle \mapsto \langle v_e, tid_e \rangle], & \text{if } v_a \in \mathbb{N} \\ M', & \text{otherwise} \end{cases} \quad T'''_I = \begin{cases} T''_I, & \text{if } v_a \in \mathbb{N} \\ T'_I, & \text{otherwise} \end{cases} \end{array}}{eval(S, Store(m, k, e_a, e_v)) \Downarrow \langle v_e, tid_e \langle R', M'', \langle T'''_I, T''_C \rangle \rangle \rangle}$$

8.3 Load

$$\frac{\begin{array}{l} eval(S, e) \Downarrow \langle v_a, tid_a, S_1 \rangle \quad S_1 = \langle R, M, \langle T_I, T_C \rangle \rangle \quad T'_C = T_C \cup \{Address(tid_a)\} \quad loc = \langle m.Identifier, v_a \rangle \\ tid_v, T'_I = newId(T_I) \quad M', T''_I = \begin{cases} M, T_I & \text{if } loc \in M \\ M[loc \mapsto \langle \top, tid_v \rangle], T'_I & \text{otherwise} \end{cases} \quad v, tid = M[loc] \end{array}}{eval(S, Load(m, k, e)) \Downarrow \langle v, tid, \langle R, M', \langle T''_I, T'_C \rangle \rangle \rangle}$$

8.4 Num

$$\frac{S = \langle R, M, \langle T_I, T_C \rangle \rangle \quad tid, T'_I = newId(T_I) \quad T'_C = CheckAddr(T_C, n, tid)}{eval(S, Num(n)) \Downarrow \langle v, tid, \langle R, M, \langle T'_I, T'_C \rangle \rangle \rangle}$$

8.5 Var

$$\frac{S = \langle R, M, \langle T_I, T_C \rangle \rangle \quad regId = ident(Var(x)) \quad v = R[regId] \quad tid = T_I[regId]}{eval(S, Var(x)) \Downarrow \langle v, tid, S \rangle}$$

8.6 Cast

$$\frac{\begin{array}{l} S = \langle R, M, \langle T_I, T_C \rangle \rangle \quad eval(S, e) \Downarrow \langle v, tid, S' \rangle \quad v_{new} = Cast(\diamond_c, v) \\ tid_{new}, T'_I = newId(T_I) \quad T'_C = T_C \cup \{Value(tid_{new})\} \end{array}}{eval(S, Cast(\diamond_c, k, e)) \Downarrow \langle v_{new}, tid_{new}, \langle R, M, \langle T'_I, T'_C \rangle \rangle \rangle}$$

8.7 ExprList

$$\frac{S_0 = S \quad eval(S_0, e_0) \Downarrow \langle v_0, tid_0, S_1 \rangle \quad \dots \quad eval(S_n, e_n) \Downarrow \langle v_n, tid_n, S_{n+1} \rangle}{eval(S, ExprList(e_0, \dots, e_n)) \Downarrow \langle List(v_0, \dots, v_n), List(tid_0, \dots, tid_n), S_{n+1} \rangle}$$

8.8 Extract

$$\frac{\begin{array}{l} eval(S, e) \Downarrow \langle v, tid, S' \rangle \quad S' = \langle R, M, \langle T_I, T_C \rangle \rangle \quad v_{new} = Extract(v, k, pos) \\ tid_{new}, T'_I = newId(T_I) \quad T'_C = T_C \cup \{Value(tid_{new})\} \end{array}}{eval(S, Extract(e, k, pos)) \Downarrow \langle v_{new}, tid_{new}, \langle R, M, \langle T'_I, T'_C \rangle \rangle \rangle}$$

8.9 FuncName

$$\overline{eval(S, FuncName(s)) \Downarrow \langle \perp, None, S \rangle}$$

8.10 ITE

$$\begin{array}{c}
\langle R, M, \langle T_I, T_C = S \rangle \rangle \quad eval(\text{filter}(S, e_c), e_t) \Downarrow \langle v_t, tid_t, S_t \rangle \quad \langle R_t, M_t, \langle T_{I_t}, T_{C_t} \rangle \rangle = S_t \\
S' = \langle R, M, \langle T_{I_t}, T_{C_t} \rangle \rangle \quad eval(\text{filter}(S', \neg e_c), e_f) \Downarrow \langle v_f, tid_f, S_f \rangle \quad \langle R_f, M_f, \langle T_{I_f}, T_{C_f} \rangle \rangle = S_f \\
v = v_t \sqcup v_f \quad tid, T'_I = newId(T_{I_f}) \quad T'_C = T_C \cup \{\text{Same}(\{tid_t, tid_f\})\} \\
\hline
eval(S, \text{ITE}(e_c, k, e_t, e_f)) \Downarrow \langle v, tid, \langle R_t \sqcup R_f, M_t \sqcup M_f, \langle T'_I, T'_C \rangle \rangle \rangle
\end{array}$$

8.11 RelOp

$$\begin{array}{c}
eval(S, e_1) \Downarrow \langle v_1, tid_1, S_1 \rangle \quad eval(S_1, e_2) \Downarrow \langle v_2, tid_2, S_2 \rangle \quad \langle R', M', \langle T_I, T_C \rangle \rangle = S_2 \\
v = evalRelOp(\diamond_r, v_1, v_2) \quad tid, T'_I = newId(T_I) \quad T'_C = T_C \cup \{\text{Value}(tid_1), \text{Value}(tid_2), \text{Value}(tid)\} \\
\hline
eval(S, \text{RelOp}(\diamond_r, k, e_1, e_2)) \Downarrow \langle v, tid, \langle R', M', \langle T'_I, T'_C \rangle \rangle \rangle
\end{array}$$

8.12 UnOp

$$\begin{array}{c}
eval(S, e) \Downarrow \langle v, tid, S_1 \rangle \quad \langle R, M, \langle T_I, T_C \rangle \rangle = S_1 \\
v_{new} = evalUnOp(\diamond_r, v) \quad tid_{new}, T'_I = newId(T_I) \quad T'_C = T_C \cup \{\text{Value}(tid), \text{Value}(tid_{new})\} \\
\hline
eval(S, \text{UnOp}(\diamond_u, k, e)) \Downarrow \langle v_{new}, tid_{new}, \langle R, M, \langle T'_I, T'_C \rangle \rangle \rangle
\end{array}$$

8.13 Undefined

$$\overline{eval(S, \text{Undefined}(k, str)) \Downarrow \langle \perp, S \rangle}$$

9 Statement Transfer Semantics

$$\langle l, S \rangle \rightarrow \langle l', S' \rangle$$

9.1 LMark

$$\frac{cmd(l) = \text{LMark}(\ell)}{\langle l, S \rangle \rightarrow \langle \text{next}(l), S \rangle}$$

9.2 Def

$$\frac{cmd(l) = \text{Def}(x, e) \quad x \in \{\text{RegVar}, \text{TempVar}, \text{PCVar}, \text{StackVar}\} \quad \text{B2R2Const}(l, x) = n \quad \langle R, M, \langle T_I, T_C \rangle \rangle = S \quad \text{regId} = \text{RegId}(x) \quad \text{tid}, T'_I = \text{newId}(T_I)}{\langle l, S \rangle \rightarrow \langle \text{next}(l), \langle R[\text{regId} \mapsto n], M, \langle T'_I[\text{regId} \mapsto \text{tid}], T_C \rangle \rangle \rangle}$$

$$\frac{cmd(l) = \text{Def}(x, e) \quad \text{B2R2Const}(l, x) \text{ is not Const} \quad eval(S, e) \Downarrow \langle v, \text{tid}, S_1 \rangle \quad \langle R, M, \langle T_I, T_C \rangle \rangle = S_1 \quad \text{regId} = \text{RegId}(x)}{\langle l, S \rangle \rightarrow \langle \text{next}(l), \langle R[\text{regId} \mapsto v], M, \langle T_I[\text{regId} \mapsto \text{tid}], T_C \rangle \rangle \rangle}$$

9.3 Phi

$$\frac{cmd(l) = \text{Phi}(x, \vec{i}) \quad S = \langle R, M, \langle T_I, T_C \rangle \rangle \quad \text{phiVals}(l, S) = \{\langle v_0, \text{tid}_0 \rangle, \dots, \langle v_n, \text{tid}_n \rangle\} \quad v_\phi = v_0 \sqcup \dots \sqcup v_n \quad \text{regId} = \text{RegId}(\text{Var}(x)) \text{tid}, T'_I = \text{newId}(T_I) \quad T'_C = T_C \cup \{\text{Same}(\{\text{tid}_0, \dots, \text{tid}_n, \text{tid}\})\}}{\langle l, S \rangle \rightarrow \langle \text{next}(l), \langle R[\text{regId} \mapsto v_\phi], M, \langle T'_I[\text{regId} \mapsto \text{tid}], T'_C \rangle \rangle \rangle}$$

9.4 Jmp

$$\frac{cmd(l) = \text{Jmp}(\text{IntraJmp}(\ell))}{\langle l, S \rangle \rightarrow \langle \ell, S \rangle}$$

$$\frac{cmd(l) = \text{Jmp}(\text{IntraCJmp}(e_c, \ell_t, \ell_f)) \quad eval(S, e_c) \Downarrow \langle v, \text{tid}, S' \rangle \quad \langle R, M, \langle T_I, T_C \rangle \rangle = S' \quad T'_C = T_C \cup \{\text{Value}(\text{tid})\}}{\langle l, S \rangle \rightarrow \langle \ell_t, \langle R, M, \langle T_I, T'_C \rangle \rangle \rangle}$$

$$\frac{cmd(l) = \text{Jmp}(\text{IntraCJmp}(e_c, \ell_t, \ell_f)) \quad eval(S, e_c) \Downarrow \langle v, \text{tid}, S' \rangle \quad \langle R, M, \langle T_I, T_C \rangle \rangle = S' \quad T'_C = T_C \cup \{\text{Value}(\text{tid})\}}{\langle l, S \rangle \rightarrow \langle \ell_f, \langle R, M, \langle T_I, T'_C \rangle \rangle \rangle}$$

$$\frac{cmd(l) = \text{Jmp}(\text{InterJmp}(e)) \quad eval(S, e) \Downarrow \langle v, \text{tid}, S' \rangle \quad \langle R, M, \langle T_I, T_C \rangle \rangle = S' \quad T'_C = T_C \cup \{\text{Address}(\text{tid})\} \quad S'' = \langle R, M, \langle T_I, T'_C \rangle \rangle \quad \text{kind}(l, e) = \text{Call}(f) \quad \Sigma_F(f) = \langle \Delta_f, \text{Ret}_f \rangle \quad \text{applySummary}(\Delta_f, \text{Ret}_f, \text{args}(l), \text{rets}(l), S'') \Downarrow S'''}{\langle l, S \rangle \rightarrow \langle \text{next}(l), S''' \rangle}$$

$$\frac{cmd(l) = \text{Jmp}(\text{InterJmp}(e)) \quad eval(S, e) \Downarrow \langle v, \text{tid}, S' \rangle \quad \langle R, M, \langle T_I, T_C \rangle \rangle = S' \quad T'_C = T_C \cup \{\text{Address}(\text{tid})\} \quad S'' = \langle R, M, \langle T_I, T'_C \rangle \rangle \quad \text{kind}(l, e) = \text{Return}}{\langle l, S \rangle \rightarrow \langle \text{Exit}, S'' \rangle}$$

$$\frac{cmd(l) = \text{Jmp}(\text{InterJmp}(e)) \quad eval(S, e) \Downarrow \langle v, \text{tid}, S' \rangle \quad \langle R, M, \langle T_I, T_C \rangle \rangle = S' \quad T'_C = T_C \cup \{\text{Address}(\text{tid})\} \quad S'' = \langle R, M, \langle T_I, T'_C \rangle \rangle \quad \text{kind}(l, e) = \text{InterJump}}{\langle l, S \rangle \rightarrow \langle v, S'' \rangle}$$

$$\begin{array}{c}
cmd(l) = \text{Jump}(\text{InterCJump}(e_c, e_t, e_f)) \quad eval(S, e_c) \Downarrow \langle v_c, tid_c, S_1 \rangle \\
eval(S_1, e_t) \Downarrow \langle v_t, tid_t, S_2 \rangle \quad eval(S_2, e_f) \Downarrow \langle v_f, tid_f, S_3 \rangle \\
\langle R, M, \langle T_I, T_C \rangle \rangle = S_3 \quad T'_C = T_C \cup \{\text{Value}(tid_c), \text{Address}(tid_t), \text{Address}(tid_f)\} \quad S_4 = \langle R, M, \langle T_I, T'_C \rangle \rangle \\
kind(l, e_t) = \text{Call}(f) \quad \Sigma_F(f) = \langle \Delta_f, Ret_f \rangle \quad \text{applySummary}(\Delta_f, Ret_f, \text{args}(l), \text{rets}(l), S_4) \Downarrow S_5 \\
\hline
\langle l, S \rangle \rightarrow \langle \text{next}(l), S_5 \rangle
\end{array}$$

$$\begin{array}{c}
cmd(l) = \text{Jump}(\text{InterCJump}(e_c, e_t, e_f)) \quad eval(S, e_c) \Downarrow \langle v_c, tid_c, S_1 \rangle \\
eval(S_1, e_t) \Downarrow \langle v_t, tid_t, S_2 \rangle \quad eval(S_2, e_f) \Downarrow \langle v_f, tid_f, S_3 \rangle \\
\langle R, M, \langle T_I, T_C \rangle \rangle = S_3 \quad T'_C = T_C \cup \{\text{Value}(tid_c), \text{Address}(tid_t), \text{Address}(tid_f)\} \quad S_4 = \langle R, M, \langle T_I, T'_C \rangle \rangle \\
kind(l, e_f) = \text{Call}(f) \quad \Sigma_F(f) = \langle \Delta_f, Ret_f \rangle \quad \text{applySummary}(\Delta_f, Ret_f, \text{args}(l), \text{rets}(l), S_4) \Downarrow S_5 \\
\hline
\langle l, S \rangle \rightarrow \langle \text{next}(l), S_5 \rangle
\end{array}$$

$$\begin{array}{c}
cmd(l) = \text{Jump}(\text{InterCJump}(e_c, e_t, e_f)) \quad eval(S, e_c) \Downarrow \langle v_c, tid_c, S_1 \rangle \\
eval(S_1, e_t) \Downarrow \langle v_t, tid_t, S_2 \rangle \quad eval(S_2, e_f) \Downarrow \langle v_f, tid_f, S_3 \rangle \\
\langle R, M, \langle T_I, T_C \rangle \rangle = S_3 \quad T'_C = T_C \cup \{\text{Value}(tid_c), \text{Address}(tid_t), \text{Address}(tid_f)\} \quad S_4 = \langle R, M, \langle T_I, T'_C \rangle \rangle \\
kind(l, e_t) = \text{Return} \\
\hline
\langle l, S \rangle \rightarrow \langle \text{Exit}, S_4 \rangle
\end{array}$$

$$\begin{array}{c}
cmd(l) = \text{Jump}(\text{InterCJump}(e_c, e_t, e_f)) \quad eval(S, e_c) \Downarrow \langle v_c, tid_c, S_1 \rangle \\
eval(S_1, e_t) \Downarrow \langle v_t, tid_t, S_2 \rangle \quad eval(S_2, e_f) \Downarrow \langle v_f, tid_f, S_3 \rangle \\
\langle R, M, \langle T_I, T_C \rangle \rangle = S_3 \quad T'_C = T_C \cup \{\text{Value}(tid_c), \text{Address}(tid_t), \text{Address}(tid_f)\} \quad S_4 = \langle R, M, \langle T_I, T'_C \rangle \rangle \\
kind(l, e_f) = \text{Return} \\
\hline
\langle l, S \rangle \rightarrow \langle \text{Exit}, S_4 \rangle
\end{array}$$

$$\begin{array}{c}
cmd(l) = \text{Jump}(\text{InterCJump}(e_c, e_t, e_f)) \quad eval(S, e_c) \Downarrow \langle v_c, tid_c, S_1 \rangle \\
eval(S_1, e_t) \Downarrow \langle v_t, tid_t, S_2 \rangle \quad eval(S_2, e_f) \Downarrow \langle v_f, tid_f, S_3 \rangle \\
\langle R, M, \langle T_I, T_C \rangle \rangle = S_3 \quad T'_C = T_C \cup \{\text{Value}(tid_c), \text{Address}(tid_t), \text{Address}(tid_f)\} \quad S_4 = \langle R, M, \langle T_I, T'_C \rangle \rangle \\
kind(l, e_t) = \text{InterJump} \\
\hline
\langle l, S \rangle \rightarrow \langle v_t, S_4 \rangle
\end{array}$$

$$\begin{array}{c}
cmd(l) = \text{Jump}(\text{InterCJump}(e_c, e_t, e_f)) \quad eval(S, e_c) \Downarrow \langle v_c, tid_c, S_1 \rangle \\
eval(S_1, e_t) \Downarrow \langle v_t, tid_t, S_2 \rangle \quad eval(S_2, e_f) \Downarrow \langle v_f, tid_f, S_3 \rangle \\
\langle R, M, \langle T_I, T_C \rangle \rangle = S_3 \quad T'_C = T_C \cup \{\text{Value}(tid_c), \text{Address}(tid_t), \text{Address}(tid_f)\} \quad S_4 = \langle R, M, \langle T_I, T'_C \rangle \rangle \\
kind(l, e_f) = \text{InterJump} \\
\hline
\langle l, S \rangle \rightarrow \langle v_f, S_4 \rangle
\end{array}$$

9.5 ExternalCall

$$\begin{array}{c}
cmd(l) = \text{ExternalCall}(e, \vec{x}_{in}, \vec{x}_{out}) \quad eval(S, e) \Downarrow \langle v, tid, S_1 \rangle \\
\langle R, M, \langle T_I, T_C \rangle \rangle = S_1 \quad T'_C = T_C \cup \{\text{Address}(tid)\} \quad \langle R, M, \langle T_I, T'_C \rangle \rangle = S_2 \\
f = \text{getFunction}(v) \quad \Sigma_F(f) = \langle \Delta_f, Ret_f \rangle \quad \text{applySummary}(\Delta_f, Ret_f, \vec{x}_{in}, \vec{x}_{out}, S_2) \Downarrow S_3 \\
\hline
\langle l, S \rangle \rightarrow \langle \text{next}(l), S_3 \rangle
\end{array}$$

9.6 SideEffect

$$\begin{array}{c}
cmd(l) = \text{SideEffect}(\text{SysCall}) \quad S' = \text{applySysCall}(l, S) \\
\hline
\langle l, S \rangle \rightarrow \langle \text{next}(l), S' \rangle \\
\\
cmd(l) = \text{SideEffect}(se) \\
\hline
\langle l, S \rangle \rightarrow \langle \text{next}(l), S \rangle
\end{array}$$

10 ApplySummary

$$\frac{\Delta_f = \langle A_f, U_f \rangle \quad \text{matchArgs}(A_f, \vec{x}_{in}, S) \Downarrow S_1 \quad \text{applyUpdates}(U_f, S_1) \Downarrow S_2 \quad \text{matchReturn}(Ret_f, \vec{x}_{out}, S_2) \Downarrow S'}{\text{applySummary}(\Delta_f, Ret_f, \vec{x}_{in}, \vec{x}_{out}, S) \Downarrow S'}$$